# Data Science Critical Dependence on Computer Science & Data Architecture

**Data Science is not standalone — it sits on top of Computer Science and Data Engineering.**

## 1. Algorithms & Computation

- ML = algorithmic engineering (complexity, memory, optimization).
- High-performance code, feasible models, faster experimenting

## 2. Software Engineering Foundations

- Clean code, versioning, modularity, testing, documentation.
- Enables reproducibility and robust pipelines.

## 3. Data Architecture & Infrastructure

- DS relies on CS-designed systems:
- **Databases** (SQL/NoSQL, time-series stores…)
- **Data formats** (Parquet, Arrow, CSV, JSON, HDF5…)
- **Distributed systems** (Spark, cloud storage layers…)
- **Data warehouses & lakes** (Snowflake, BigQuery, S3-based lakes…)

## 4. Data Pipelines & Orchestration

- ETL/ELT workflows (Airflow, Prefect …)
- Real-time streaming (Kafka, Kinesis….)
- Ensures clean, reliable, timely data at scale

# Data Science Critical Dependence on Computer Science & Data Architecture

## 5. Data Quality & Governance

- Schema management, data validation, metadata, lineage.
- Data must be *accurate, consistent, complete* before ML touches it.

## 6. Storage, Archival, and Access

- Tiered storage, backups, archival policies, GDPR/retention rules.
- Efficient access patterns matter (indexing, caching, partitioning).

## 7. Data-Related Risks in ML

- **Concept drift** (relationships change)
- **Data drift** (distribution shifts)
- **Feature decay** (pipelines break)
- **Data leakage** (future info leaks)
- CS + monitoring systems are required to detect & fix these.

## 8. Productionisation & Lifecycle

- CI/CD for ML, MLOps, deployment patterns (REST, batch, streaming).
- Monitoring performance, drift, latency, resource usage.

# Core Language Concepts - Interpreted vs. Compiled

## 1. Interpreted Languages (Python, R, JavaScript, Julia)

- Code is executed **line-by-line** by an interpreter
- Flexible, dynamic, great for rapid development
- Slower than compiled languages — but fine for DS when paired with fast libraries (NumPy, PyTorch)
- Easy debugging and experimentation → perfect for notebooks

## 2. Compiled Languages (C, C++, Rust, Java)

- Code is translated into **machine code** before running
- Much faster, more memory-efficient
- Stricter, harder to prototype

# Core Language Concepts - • Procedural vs. Functional vs. Declarative

## 1. Procedural Languages (C, early Python style)

- Program is a sequence of explicit steps
- Control flow is everything: loops, conditions, state changes
- Simple mental model, great for small scripts

**Challenges:**

- Can become messy and unstructured as complexity grows
- Much code duplication of repetitive tasks

## 2. Functional Programming (modern Python, Scala, F#, data pipelines)

- Programs are built from **functions**, not steps
- Emphasizes **pure functions** (no side effects, **no shared state**)
- Great for data transformations, pipelines, and parallel computation
- Reusable

## 3. Declarative Languages (SQL, HTML, Prolog, dataframe APIs)

- You describe **what** you want, not **how** to compute it
- Under the hood, the engine optimizes execution
- Often used for data queries, configuration, analytical expressions

**Example:**

- SELECT avg(price) FROM trades — no loops, no steps, just intent

# Core Language Concepts - Object-Oriented Programming (OOP)

**OOP structures code around  i.e. data and behaviour, therefore easier to manage complexity in larger systems**

**OOP**

- Code is organized into **classes** (templates) and **objects** (instances).
- A class defines:
  - ✓ **Attributes** → data/state
  - ✓ **Methods** → functions acting on that data

## Pros

- **Encapsulation:** keep data + logic together
- **Reusability:** classes can be reused across projects
- **Modularity:** easier to maintain, extend, and test
- **Inheritance:** build new classes from existing ones
- **Pseudo standard:** ML frameworks (scikit-learn, PyTorch) are OOP at the core

## Applications

- Model wrappers (scikit-learn estimators)
- Simulation frameworks (e.g. state-space models)
- Reusable feature engineering pipelines

**Complexity**

- Harder for beginners — abstract concepts (inheritance, polymorphism)
- Over-engineering risk — too many small classes = confusion
- State management can get messy if not carefully designed

# Python Language Fundamentals for Data Science

## 1. Core Python Syntax & Control Flow

- Variables & types (int, float, str)
- **Loops**: for, while
- **Conditionals**: if / elif / else
- **Functions**: def, arguments, return values
- List comprehensions (Pythonic iteration)

## 2. Essential Data Structures

- **Lists** – ordered collections
- **Tuples** – immutable groups
- **Dictionaries** – key/value lookups
- **Sets** – unique elements
- Choosing the right structure → faster, cleaner code

## 3. Numerical & Array Operations

- **NumPy arrays**
- Vectorized math
- Matrices & linear algebra
- Broadcasting rules
- Efficient computation for ML & time-series

# Python Language Fundamentals for Data Science

## 4. Data Access & Handling

- Reading/writing: CSV, JSON, Parquet, Excel
- File I/O (open, with)
- APIs (HTTP requests), databases (SQL connectors)

## 5. DataFrames (pandas, polars) (distributed: dask, spark)

- Series vs DataFrame
- Filtering, joining, grouping
- Missing values, indexing, datetime operations
- Core tool for transforming raw data → model-ready data

## 6. Plotting & Visualization Basics

- Matplotlib, Seaborn
- Line, scatter, histograms, heatmaps
- Quick EDA for understanding data patterns

# Functional Example to OOP

```python
def clean(x):
    return x.strip().lower()

def tokenize(x):
    return x.split()

def count_tokens(tokens):
     return len(tokens)

# Pipeline
text = "Hello World"
result = count_tokens(tokenize(clean(text)))
```

```python
class TextProcessor:
    def __init__(self, text):
        self.text = text

    def clean(self):
        self.text = self.text.strip().lower()
        return self

    def tokenize(self):
        self.tokens = self.text.split()
        return self

    def count_tokens(self):
        return len(self.tokens)

processor = TextProcessor("Hello World")
result = processor.clean().tokenize().count_tokens()
```

- **Functional:** stateless, direct, minimal overhead
- **OOP:** structured, reusable, good for complex systems

# Core Concepts - Multithreading, Multiprocessing, and GPU's

## Multithreading

- Multiple **threads** inside one Python process
- All threads share the same memory
- Lightweight and good for I/O-bound tasks ,network calls
- **Limited by the GIL** - one thread can execute Python at a time
- Not for CPU-heavy numeric loops written in pure Python

### Use cases:

- Downloading data
- Calling APIs
- Reading/writing many files
- Handling I/O concurrency

## Multiprocessing

- Multiple **processes**, each with its own Python interpreter
- Each process has its own memory - avoids GIL completely
- Allows true parallelism across CPU cores
- More overhead (process creation, data serialization)
- Best for CPU-bound tasks that need real parallel computation

### Use cases:

- Numerical computation
- Heavy data transformations
- Simulations
- Feature engineering across large chunks
- Anything that maxes out CPUs

## GPU

- Thousands of lightweight cores designed for **massively parallel math**
- Best for tensor operations: matrix multiplications, deep learning
- Requires CUDA(***NVIDIA***)/ROCm(***AMD***) and GPU-friendly libraries (PyTorch, TensorFlow, JAX, CuPy)
- **Not general-purpose like CPUs — optimized for linear algebra**
- Deep learning
- Vectorized math
- Large matrix operations
- Acceleration of numerical models